

Project Motivation

Oncology departments handle a vast number of patients daily, generating large volumes of complex and unstructured clinical data. This reality, common in many healthcare systems, is characterized by handwritten free-text notes, leading to a highly unstandardized set of clinical records. This poses significant challenges in information management, as the critical data needed for patient monitoring and research support is locked within difficult-to-process formats.

Oncologists, who already operate under high cognitive and emotional demands, face an immense administrative burden from the manual extraction and analysis of this data. This contributes to high burnout rates and operational inefficiency. A critical consequence of this data chaos is the persistent under-enrolment of eligible patients in clinical trials. When patient data from clinical diaries and notes cannot be effectively matched with the structured eligibility criteria of trials, institutions face financial penalties and lose valuable opportunities to advance research.

This project aims to address these challenges by leveraging Large Language Models (LLMs), which present a promising tool to overcome this long-standing issue. Unlike previous solutions that were primarily applied to structured data, LLMs can "understand" human text and generate coherent responses, even when faced with free text, varied writing styles, and errors. By using LLMs to automate the extraction, normalization, and analysis of unstructured oncology data, this project seeks to infer eligible patients for clinical trials. The ultimate goals are to reduce the administrative load on healthcare professionals, enhance clinical decision support, and improve patient recruitment, thereby contributing to a more efficient and data-driven oncology healthcare ecosystem.

Requirements

1. Identification of Eligible Patients for Clinical Trials

1. The system employs a LLM-based engine that extracts inclusion and exclusion criteria from clinical trial documentation and converts them into computational logic (e.g. Age>18);
2. These logical rules are then automatically compared with the structured patient database, identifying candidates who meet all eligibility conditions.

2. Monitoring the Quality of Care

1. The system provides survival and outcome statistics based on treatment data, enabling clinicians to assess the effectiveness of therapies and monitor long-term patient outcomes;

2. Relevant data fields for these analyses include: start and end dates of medication intake, diagnosis date, date of death or last consultation, and whether disease progression occurred
3. Automated Structured Data Extraction
 1. A LLM automatically extracts structured information from free-text clinical diaries;
 2. The extracted fields align with those defined by Dr. Judy's: Age, Date of Birth, Chemotherapy Protocol, Diagnosis, Staging and Molecular Status;
 3. Since the clinical text may contain typographical or linguistic inconsistencies, the system integrates a language normalization process to ensure data consistency and reliability
 4. In the end of this process of data extraction each structured patient profile should be store in a database for further use for the patient eligibility phase.

Workflow of the patient

When a patient arrives at the hospital, the first step is to undergo an emergency diagnosis followed by/or an evaluation within primary health care, where an initial diagnosis is established, and the staging status is determined. After this, a therapeutic decision meeting takes place. Based on the patient's staging, the clinical pathway may diverge into several possible treatment routes.

Staging is a system used by clinicians to describe the extent and spread of cancer at the time of diagnosis. It assesses the size of the tumour, its depth of invasion, whether it has spread to nearby tissues or lymph nodes, and if it has metastasized to distant parts of the body. This classification helps guide treatment planning and decision-making, aligning patients with the most suitable therapeutic options. Although patients may share the same stage, their clinical pathways can differ depending on the type and characteristics of the tumour.

The most widely used system for staging is the TNM classification, which consists of three main components:

- T (Tumour): Describes the size and extent of the primary tumour (graded from T_0 to T_4).
- N (Nodes): Indicates whether the cancer has spread to nearby lymph nodes (graded from N_0 to N_3).
- M (Metastasis): Reflects whether the cancer has spread to distant parts of the body (graded from M_0 to M_1).

PET/CT scans play a crucial role in the staging process by providing both metabolic and anatomical insights, enabling a precise assessment of tumour size, location, and spread.

- PET (Positron Emission Tomography): Measures metabolic activity using a radioactive tracer (commonly a glucose analogue) that accumulates in metabolically active cells. Cancer cells, due to their higher metabolic rates, appear as bright or “hot” spots on the scan.

- CT (Computed Tomography): Produces detailed cross-sectional images of the body's internal structures, revealing the exact shape, position, and relationship of organs and tissues.

By combining PET and CT images, clinicians obtain a comprehensive three-dimensional view that correlates metabolic activity with precise anatomical locations. This integration significantly enhances the accuracy of cancer detection, staging, and treatment planning.

Stages of Cancer

Stage

1

Early Stage

- A small, invasive mass or tumor has been found.
- No spread to lymph nodes or other tissues.
- Sometimes called early-stage or “localized” cancer.

Stage

2

Localized

- Cancer has started to affect nearby tissue.
- Mass may have grown in size.
- Spread to lymph nodes near the mass.

Stage

3

Regional Spread

- Cancer affects more surrounding tissue.
- Mass may have grown in size.
- Spread to distant lymph nodes away from the mass.

Stage

4

Distant Spread

- Cancer has spread to other tissues or organs beyond the region where it originated.
- Sometimes called advanced or “metastatic” cancer.

For more cancer topics and advice visit:

www.shine365.marshfieldclinic.org/category/cancer-care

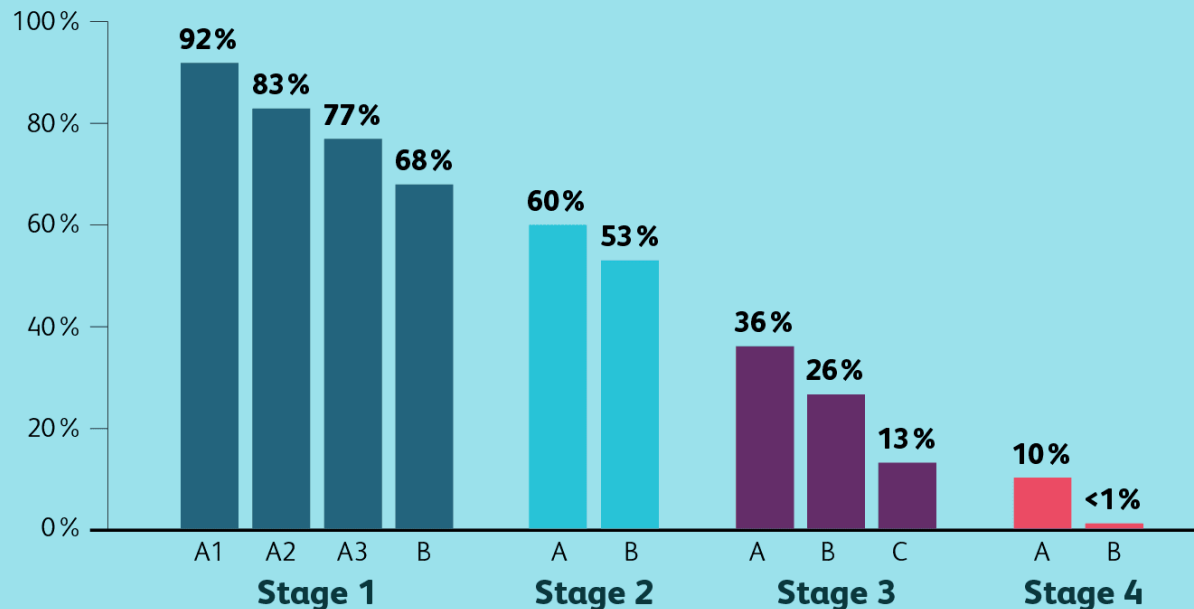
To learn about oncology specialty services at Marshfield Clinic:

www.marshfieldclinic.org/specialties/cancer-care



**Marshfield Clinic
Health System**

Non-Small Cell Lung Cancer: 5-Year Survival Rates



verywell

Source: American Cancer Society 2017

Workflow of the LLM

The LLM workflow for the current project is divided into two main phases:

- Phase 1: Extraction of Hyper-Relevant Parameters
- Phase 2: Application of Clinical Trial Criteria

Phase 1 - Hyper-Relevant Parameter Extraction

The goal of this first phase is to transform free-text, unstructured clinical information into normalized and structured data.

The input consists of clinical diaries written in free text. The process follows these steps:

1. Text Pre-processing: Retrieve the source documents from SClinico and extract their textual content.
2. Information Extraction: The LLM receives the text document along with a carefully crafted prompt. It is instructed to identify and extract hyper-relevant clinical parameters.
3. Structured Output Generation: The LLM's output, formatted in JSON, is stored in a database for subsequent use.

Upon completion of this phase, a structured database containing the key clinical parameters for each patient is obtained.

Phase 2 - Application of Clinical Trial Criteria

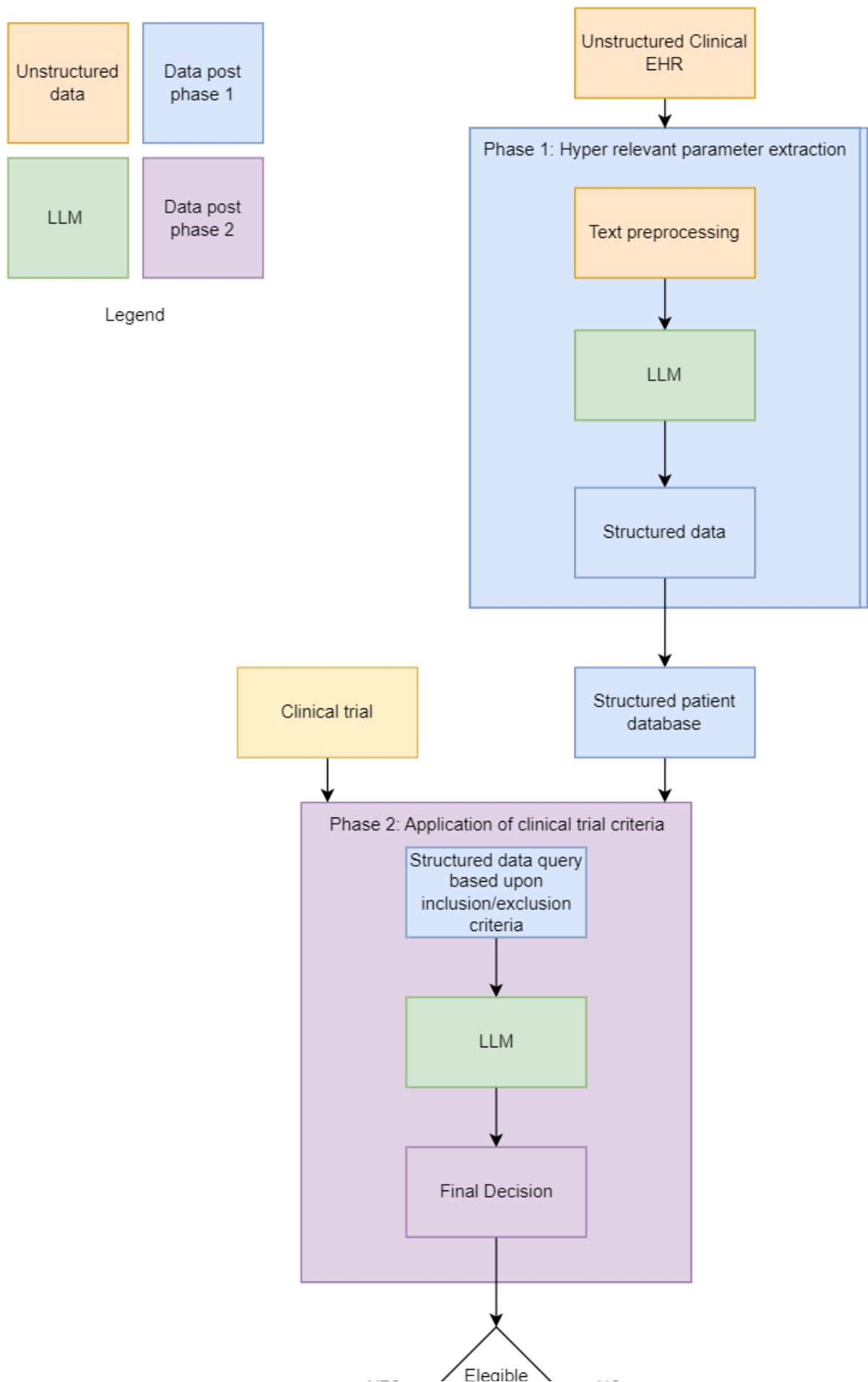
The goal of this second phase is to evaluate the structured patient profiles against the inclusion and exclusion criteria of a specific clinical trial.

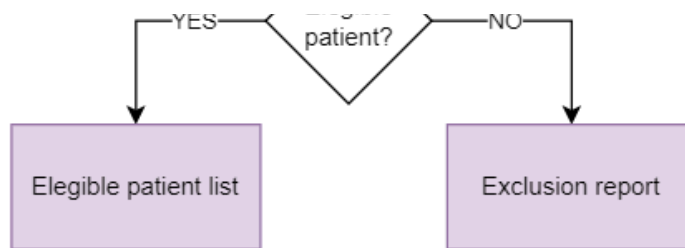
The inputs are the structured patient database produced in Phase 1 and the corresponding clinical trial documentation.

- The inclusion and exclusion criteria are extracted from the clinical trial and translated into computational logic (e.g., Age > 18).
- The LLM then evaluates each patient according to these criteria.
- The output produced by the model is a categorical assessment: “Yes”, “No”, or “Inconclusive.”

After this process, two key artifacts are generated:

- Eligible Patient List: A list of patients who meet all the trial criteria.
- Exclusion Report: A detailed report specifying, for each excluded patient, which criteria were not satisfied.





Background

Symbolic and Connectionist AI

Symbolic AI systems are explicitly designed and programmed, that is, human experts encode their domain knowledge directly into the system. The model receives a **set of rules** and an input, applying those rules to generate an appropriate output.

For example, in a grammar-based system:

Rule set:

- Sentence → Subject + Predicate
- Subject → Article + Noun
- Article → "the" | "a"
- Predicate → Verb + Object

Input: "The cat chases the mouse"

Application of rules:

- "The" → Article
- "cat" → Noun → together form the Subject
- "chases" → Verb
- "the" → Article
- "mouse" → Noun → together form the Predicate

Thus, the system determines that the sentence is grammatically correct.

However, this rule-based approach relies on **fixed and absolute rules**, which makes it difficult to apply to real-world situations where such absolutes rarely exist. Moreover, as problem complexity increases, the number of required rules grows exponentially, making maintenance and scalability impractical.

To overcome these limitations, **probabilistic reasoning** was introduced, along with the development of extensive rule sets designed by experts, giving rise to Expert Systems.

In contrast, the Connectionist approach is inspired by the structure and functioning of the human brain, particularly the interconnected network of neurons, hence the term connectionist. This paradigm relies on **statistics** and **information theory** rather than explicitly defined rules.

Connectionism represents the foundation of Machine Learning (ML) and, consequently, of Large Language Models (LLMs). In modern ML, a task is framed as a **statistical inference problem** based on a large dataset.

There are three main learning paradigms:

- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning, in which the task is formulated as an *objective function* to be optimized.

According to Occam's Razor, simpler models tend to generalize better. Therefore, a model should include only the **minimum number of parameters** necessary to fit the dataset effectively.

Terms such as Deep Learning and Connectionism refer to the use of Neural Networks as the underlying computational structure.

Nonetheless, the Interpretability Problem remains one of the main challenges in deep learning. The large number of parameters in neural networks makes it extremely difficult to understand how the model reaches its conclusions or why a given input leads to a particular output.

Language Models

Large Language Models (LLMs) are statistical models designed to encode a probability distribution over sequences of words.

Formally, the probability of a sentence consisting of c words can be represented as:

$$P(W_1, W_2, \dots, W_c)$$

where W_1 denotes the first word in the sequence.

From Simple Models to Probabilistic Language Understanding

The simplest case, a 1-gram (unigram) model, assumes that each word is independent of all others. In such a case, the probability of the entire sentence is simply the product of the individual word probabilities:

$$P(W_1, W_2, \dots, W_L) = \prod_{i=1}^L P(W_i)$$

where

$$P(W) = \frac{\text{Number of occurrences of } W \text{ in the corpus}}{\text{Total number of words in the corpus}}$$

Here, the corpus refers to a large collection of text used for estimating these probabilities.

However, modern LLMs are generative models, which means they can *sample* from this learned probability distribution to generate new sequences.

Given a context, i.e., a sequence of words, the model estimates the probability of the most likely next word:

$$P(\text{next word}|\text{context})$$

Examples of Prediction Tasks

- Masked Word Prediction

Example:

$$P(\text{"went"}|\text{"The cat [BLANK] outside"}) = 0.5$$

$$P(\text{"sat"}|\text{"The cat [BLANK] outside"}) = 0.2$$

$$P(\text{"is"}|\text{"The cat [BLANK] outside"}) = 0.1$$

Each probability contributes to a total of 1 (100%).

The input is "The cat ... outside" and the output is the missing token.

- Next Word Prediction

$$P(\text{"outside"}|\text{"The cat went"}) = 0.7$$

$$P(\text{"home"}|\text{"The cat went"}) = 0.2$$

$$P(\text{"to"}|\text{"The cat went"}) = 0.1$$

By sampling from this distribution and appending the predicted word to the existing sequence, the model can generate text one token at a time:

$$P(W_{n+1}|W_1, W_2, \dots, W_n)$$

Example:

$$P(W|\text{"the cat"}) \rightarrow W = \text{"went"}$$

$$P(W|\text{"the cat went"}) \rightarrow W = \text{"outside"}$$

$$P(W|\text{"the cat went outside"}) \rightarrow W = \text{"and"}$$

This iterative use of the model's own output as the next input defines an autoregressive model, represented by:

$$\begin{aligned} P(\text{"the cat went outside"}) &= P(\text{"the"}) \\ &\times P(\text{"cat"}|\text{"the"}) \\ &\times P(\text{"went"}|\text{"the cat"}) \\ &\times P(\text{"outside"}|\text{"the cat went"}) \end{aligned}$$

Evaluating Language Models

To assess how well a language model approximates the true probability distribution of a corpus, the standard metric used is **cross-entropy loss**:

$$L = -\frac{1}{N} \prod_{i=1}^{N-n} \log P(W_{i+n} | W_1, W_{i+1}, \dots, W_{i+n-1})$$

The summation captures the “surprise” of each word in the training corpus, except for the first n words that serve as context.

The term $\frac{1}{N}$ averages the total surprise across the dataset.

The logarithm penalizes confident but incorrect predictions:

for example, if $P = 0.01$, then $\log(P) = -4.6$; if $P = 0.99$, then $\log(P) = -0.01$.

The negative sign transforms this measure into a **loss function** to be minimized during training.

$$P(W_{i+n} | W_1, W_{i+1}, \dots, W_{i+n-1})$$

represents the probability the model assigns to the next word.

For example, if $i = 1$ and $n = 3$, for the sentence “The cat hunts the mouse”, we would be computing $P(\text{“the”} | \text{“The cat hunts”})$.

Training Process

The training process of a language model can be summarized as follows:

1. A sequence of words is provided to the model, which predicts the next word and assigns probabilities to all vocabulary tokens.
2. The **loss** LLL is calculated to measure how far the prediction is from the true target.
3. The **error** is propagated backward through the network (*backpropagation*).
4. Model parameters are updated via **gradient descent**.
5. The process repeats iteratively until convergence.

Simpler Generative Language Models (N-gram)

Through the statistical analysis of neighbouring words, it is possible to capture quantitative properties, and even aspects of meaning, of words.

The simplest expression of this idea is the co-occurrence matrix.

Language models operate on **tokens** rather than whole words. For example:

- “Supersymmetrization” → “Super”, “Symmetry”, “ization”

This process, called tokenization, helps models to better understand the structure and meaning of language.

Co-occurrence Matrix

Let w be the set of words in a corpus, and let $|w|$ denote its cardinality.

We define the N-gram co-occurrence matrix M_N , which is of size $|w| \times |w|$.

Each row and column represents a word, and each entry (w, w') counts the number of N-grams in the corpus that contain both words.

Example corpus:

“O gato caça o rato. O cão guarda a casa”

Then:

$$w = [O, \text{gato}, \text{caça}, \text{rato}, \text{cão}, \text{guarda}, a, \text{casa}]$$

To create a 3-gram co-occurrence matrix, we extract trigrams from the corpus.

For instance, checking for “o” and “gato”:

- “O gato caça” $\rightarrow$ contains both
- “gato caça o” $\rightarrow$ contains both
- “caça o rato” $\rightarrow$ does **not** contain both

The resulting matrix might look as follows:

	o	gato	caça	rato	cão	guarda	a	casa
o	-	2	2	3	2	1	0	0
gato	2	-	2	0	0	0	0	0
caça	2	2	-	1	0	0	0	0
rato	3	0	1	-	1	0	0	0
cão	2	0	0	1	-	2	1	0
guarda	1	0	0	0	2	-	2	1
a	0	0	0	0	1	2	-	1
casa	0	0	0	0	0	1	1	-

Once this matrix is built, **word embeddings** can be derived simply by taking the corresponding column (or row) vector of the word:

$$\text{“gato”} \Rightarrow [2, 0, 2, 0, 0, 0, 0, 0]$$

This vector provides a distributional representation of the word.

However, such vectors tend to have very high dimensionality.

To address this, dimensionality reduction techniques such as Principal Component Analysis

(PCA) are applied.

The resulting lower-dimensional vectors are then fed into the model.

Predicting the Next Word

To predict the most probable next word, the model outputs a vector v , and the system selects the word w that maximizes the inner product between v and its embedding:

$$v \cdot \iota(w)$$

where $\iota(w)$ is the embedding of word w derived from the co-occurrence matrix.

The vector v itself is generated by a function F , which may be implemented as a Neural Network, Transformer, or other architectures.

To convert the similarity scores into a probability distribution, the Boltzmann distribution is applied:

$$P(w) = \frac{e^{\beta v \cdot \iota(w)}}{\sum_{w'} e^{\beta v \cdot \iota(w')}}.$$

- $v \cdot \iota(w)$: measures similarity between the word and the predicted
- $e^{\beta v \cdot \iota(w)}$: transforms scores into positive values and amplifies differences
- $\sum_{w'} e^{\beta v \cdot \iota(w')}$: ensures probabilities sum to 1

Semantic Properties of Word Embeddings

It can be observed that word embeddings capture semantic relationships. For example:

$$\iota(\text{king}) - \iota(\text{man}) + \iota(\text{woman}) \approx \iota(\text{queen})$$

This phenomenon arises from the statistical regularities of word co-occurrences, and requires a large corpus to emerge clearly.

Generalized Autoregressive L-gram Model

From these ideas, we can define a general autoregressive L-gram model:

1. Map the L input words w_i to their embedding vectors $\iota(w_i)$.
2. Apply the function F to this list of vectors to produce a prediction vector v .
3. Use the Boltzmann distribution to obtain a probability distribution over all possible words.

Regarding the function F :

- A simple Feedforward Network can be used, but such networks expect a single input, whereas we have L vectors, one per word.

- One possible solution is to concatenate these vectors into a single input.

However, this approach proved limited, as transformer architectures later emerged and surpassed it.

A major challenge is that predicting the next word requires remembering previous context.

A simple improvement involves adding memory, which leads to Recurrent Neural Networks (RNNs).

RNNs maintain a state vector s_i that updates with each word:

$$(v_{i+1}, s_{i+1}) = F(s_i, v_i, v_{i-1}, v_{i-2}, \dots, v_{i-k+1})$$

- (v_{i+1}, s_{i+1}) : output of F ; v_{i+1} predicts the next word, and s_{i+1} is the updated memory state
- $F(s_i, v_i, v_{i-1}, \dots, v_{i-k+1})$: input to F ; s_i encodes memory so far, while $v_i, v_{i-1}, \dots$ provide immediate context within a window of k words

Recipe for a Large Language Model

A **Transformer** can be defined as a function F that takes a list of vectors as input and produces an output vector.

This represents a generalization of Feedforward Networks (FFNs), with an important distinction: Transformers account for permutation symmetry, meaning they can handle sequences where the order of inputs matters.

A Transformer is composed of two key types of layers:

1. Feedforward Layers

A Feedforward Neural Network (FFN) is applied independently to each embedding vector. For each input vector u_i , the model applies a learned transformation to produce an output vector:

$$v_i = F_{\text{FFN}}(u_i)$$

2. Attention Mechanism

The Attention mechanism allows each output vector to “attend” to, or focus on, other input vectors in the sequence.

This enables the model to determine which parts of the input are most relevant for each word at a given moment.

The output vector v_i is computed as a weighted sum of all input vectors u_j , each multiplied by a learnable linear transformation matrix B :

$$\{u_i\} \rightarrow \{v_i = w \sum_{j=1}^i\}$$

Where,

- v_i , is the output vector.
- w , is the transformation matrix.
- $c_{i,j}$, are the attention weights.
- u_j , are the input vectors.

$$c_{i,j} \equiv \frac{e^{u_i \cdot B \cdot u_j}}{\sum_{j=1}^i e^{u_i \cdot B \cdot u_j}}$$

The attention weights $c_{i,j}$ determine how much attention the vector at position i gives to the vector at position j .

These weights are computed using a softmax function, which converts numerical scores into normalized probabilities, ensuring that the sum of all $c_{i,j}$ for a given i equals 1.

The matrix B is a learned parameter during training. It allows the model to learn which relational aspects between vectors are most relevant.

- When $B = I$, the dot product measures only similarity.
- When B is learnable, it can capture more complex relationships, such as those between verbs and subjects, even when they are not semantically similar.

3. Attention Heads

Transformers use multiple attention heads operating in parallel rather than relying on a single mechanism.

Each head has its own learnable matrices B and W , and produces output vectors of smaller dimension q (compared to the original embedding dimension p).

If there are H heads, their outputs are concatenated, resulting in an overall embedding dimension:

$$p = H \times q$$

Analogy:

Each attention head may focus on a different aspect of the input:

- Head 1 $\rightarrow$ object colour;
- Head 2 $\rightarrow$ shape
- Head 3 $\rightarrow$ spatial relationships
- Head 4 $\rightarrow$ texture

This multi-head attention enables the model to integrate diverse types of information simultaneously.

4. Positional Encodings

A pure attention mechanism is invariant to input order, meaning it cannot inherently distinguish between permutations of the same words.

To address this, Transformers introduce Positional Encodings, numerical representations that encode the position of each word in the sequence.

These encodings are generated using trigonometric functions:

$$(e_{2i-1}, e_{2i}) = \left(\cos \frac{\text{position}}{10000^{\frac{2i}{d_{\text{pos}}}}}, \sin \frac{\text{position}}{10000^{\frac{2i}{d_{\text{pos}}}}} \right), \quad i \in \{1, \dots, \frac{d_{\text{pos}}}{2}\}$$

Where:

- $2i$, index within the vector
- d_{pos} , dimensionality of the positional embedding (equal to the word embedding dimension, e.g. $d = 512$)
- $\frac{d_{\text{pos}}}{2}$, ensures the sine and cosine functions alternate along half the embedding dimension

This mechanism provides positional context, ensuring that word order affects meaning, for instance:

“The cat hunts” $\neq$ “Hunts the cat”

Although both sentences contain the same words, their meanings differ due to positional encoding.

Each word’s embedding is summed with its positional vector, e.g.:

- Word embedding: $[0.3, 0.9, 0.1, 0.4]$
- Positional encoding: $[0.84, 0.54, 0.01, 1.00]$
- Combined input: $[1.14, 1.44, 0.11, 1.40]$

Thus, even repeated words at different positions in the text will receive unique positional embeddings, enabling the model to preserve sentence structure and meaning.

Attention Is All You Need

Model Architecture

The Transformer architecture is composed of two main components: an encoder and a decoder. The encoder maps a sequence of symbolic representations

$$(x_1, \dots, x_n)$$

(e.g., words) into a sequence of continuous representations

$$Z = (Z_1, \dots, Z_n)$$

The decoder then generates a sequence of output symbols

$$(y_1, \dots, y_m)$$

one element at a time, consuming previously generated symbols as additional input, making the model autoregressive.

Encoder

The encoder consists of a stack of $N = 6$ layers, each containing two sub-layers:

1. A multi-head self-attention mechanism
2. A feedforward neural network (FFN)

A layer normalization step is applied between sub-layers to stabilize and accelerate training.

Decoder

The decoder also consists of $N = 6$ layers.

In addition to the two sub-layers found in the encoder, the decoder introduces a third sub-layer that performs multi-head attention over the encoder's output.

Layer normalization is again applied between all sub-layers.

A key component in the decoder is the Masked Multi-Head Attention mechanism, which prevents each position from attending to subsequent (future) words.

This ensures that predictions for position i depend only on positions $< i$.

Example:

- Input (to the decoder):
[< s >, "The", "cat", "chases", "the"]
- Target sequence:
["The", "cat", "chases", "the", "mouse"]

Time Step	Decoder Input	Target Output
1	[< s >]	"The"
2	[< s >, "The"]	"cat"

This design ensures that each word at position i can only “see” previous words, maintaining the causal structure required for autoregressive generation.

The Attention Mechanism

The Attention mechanism maps a query and a set of key-value pairs to an output, computed as a weighted sum of the values.

Each weight represents how compatible the query is with the corresponding key.

Example context:

- Sentence: “The cat chases”
- Query: “chases”
- Keys: [“The”, “cat”, “chases”]

Mathematically:

$$Q = W_q \times \text{embedding}(\text{“persegue”})$$

$$K = W_k \times [\text{embedding}(\text{“The”}), \text{embedding}(\text{“cat”}), \text{embedding}(\text{“chases”})]$$

$$V = W_v \times [\text{embedding}(\text{“The”}), \text{embedding}(\text{“cat”}), \text{embedding}(\text{“chases”})]$$

This allows the model to build contextualized and dynamic representations, capturing:

- Long-range dependencies: (“The cat ... chases ...”)
- Context-dependent meanings: (“The bat is a nocturnal animal” vs. “He swung the bat and hit a home run”)

Because the matrices W_q , W_k , and W_v are learnable parameters initialized randomly, the model can learn diverse linguistic relations in parallel during training.

Scaled Dot-Product Attention

The Scaled Dot-Product Attention function is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where Q and K are matrices with dimension d_k .

- QK^T : computes similarity scores between each query and key (compatibility matrix).
- $\text{softmax}(\cdot)$: converts these scores into normalized probabilities.

This function processes multiple queries simultaneously, enabling efficient parallel computation.

Applications of Attention in the Transformer

There are three types of attention mechanisms in the Transformer model:

1. Encoder–Decoder Attention

- Queries (Q): come from the decoder (previous decoder output).
- Keys (K) and Values (V): come from the encoder’s final output.
- Example:
 - Encoder input: “The cat chases the mouse”
 - Decoder output so far: “The cat chases”
 - To generate “mouse”:
 $Q = \text{“chases” (Decoder)}$, $K, V = [\text{“The”, “cat”, “chases”, “the”, “mouse”}]$ (Encoder)

2. Self-Attention in the Encoder

- Q, K, V all come from the same source, the encoder’s previous layer output.
- Each word can attend to **all words** in the input.
- Example:
 - Input: “The cat chases the mouse”
 - For “chases”:
 $Q = \text{“chases”}$, $K, V = [\text{“The”, “cat”, “chases”, “the”, “mouse”}]$

3. Masked Self-Attention in the Decoder

- Q, K, V all come from the decoder.
- Future words are masked to prevent information leakage (“cheating”).
- Example:
 - Decoder: $[\text{“The”, “cat”, “chases”, “the”, “mouse”}]$
 - For “chases”, the model can only attend to:
 $[\text{“The”, “cat”, “chases”}]$

State of the Art

Extraction of Unstructured Clinical Information through LLMs

The extraction of meaningful information from unstructured clinical data such as electronic health records (EHRs) is a fundamental challenge in medical informatics. Recent studies have explored the potential of large language models (LLMs) to automate and enhance this process, focusing on diverse tasks including entity recognition, summarization, and classification of clinical text. The following works provide an overview of the methods, results, and limitations associated with these approaches.

Evaluating LLM Training and Prompting Strategies

The paper “Evaluating approaches of training a generative large language model for multi-label classification of unstructured electronic health records” investigated various training and

prompting methods for improving LLM accuracy in clinical contexts. Techniques such as zero-shot and few-shot prompting were tested in combination with Parameter-Efficient Fine-Tuning (PEFT) and Retrieval-Augmented Generation (RAG). The results showed that PEFT significantly increased performance in both zero-shot and few-shot learning. After PEFT, zero-shot learning achieved performance comparable to few-shot learning across all tasks. When neither PEFT nor RAG was applied, few-shot learning consistently outperformed zero-shot learning, while RAG improved results only in the few-shot setting. Overall, PEFT proved to be the most impactful technique, offering strong guidance for future fine-tuning approaches to unstructured clinical text analysis.

Comparative Evaluation of LLMs for Information Extraction

The study “A comprehensive evaluation of large language models for information extraction from unstructured electronic health records in residential aged care” proposed a holistic evaluation framework for information extraction (IE) from free-text clinical notes. It systematically assessed different LLMs on Named Entity Recognition (NER) and summarization tasks using real-world unstructured data. Results indicated that while LLMs achieved strong traditional metrics, maintaining contextual relevance and robustness remained challenging. Health-domain-specialized models performed slightly better than general-purpose ones, and approaches that incorporated RAG yielded superior outcomes. These findings highlight the need for domain adaptation and hybrid architectures to ensure reliable performance in sensitive medical settings.

Efficiency of LLMs Compared to Manual Review

In “Large language models for efficient medical information extraction,” the efficiency of ChatGPT 3.5 was compared against manual data review for extracting clinical information from unstructured records. Despite being an out-of-the-box model, ChatGPT exhibited high specificity and sensitivity and proved capable of identifying most relevant clinical concepts. However, it struggled with nuanced semantic information concerning specific conditions. The study concluded that LLMs can outperform traditional NLP systems that often require extensive training and manual effort. With targeted fine-tuning and structured extraction rules, models such as ChatGPT could further advance medical information extraction efficiency and reliability.

Large-Scale Model Benchmarking for Data Extraction

The work “Large language models for data extraction from unstructured and semi-structured electronic health records: a multiple model performance evaluation” assessed the performance of eighteen different LLMs using synthetic medical notes validated by specialists. The evaluation covered four entity extraction tasks and five binary classification tasks under a zero-shot setting. Output consistency was measured across repeated prompt iterations. Advanced LLMs, including Claude 3.0 Opus, Claude 3.0 Sonnet, Claude 2.0, and GPT-4, achieved

significantly superior performance compared to baseline models such as RoBERTa, particularly in entity extraction. This research demonstrated that modern LLMs can reliably extract structured information from EHRs and substantially reduce clinicians' data processing workload.

Extraction of Social Determinants of Health (SDoH)

The study "Scalable information extraction from free text electronic health records using large language models" explored whether open-source LLMs could effectively extract Social Determinants of Health (SDoH) from clinical text. Two pipelines, Default and Refined, were implemented, with the latter incorporating prompt engineering to mitigate hallucinations. Manual annotations by human reviewers were used as gold standards. Results showed that LLMs outperformed the traditional pattern-matching baseline, particularly in cases where SDoH were explicitly mentioned. The Refined pipeline successfully reduced hallucination rates and improved model conservativeness. Open-source models such as openchat_3.5 demonstrated both efficiency and scalability, with quantized variants proving robust in resource-limited environments. This study emphasized the value of prompt design for optimizing accuracy in unstructured text extraction without fine-tuning.

Oncology Information Extraction with Expert-Annotated Data

The "CORAL: Expert-Curated Oncology Reports to Advance Language Model Inference" paper introduced a new annotated dataset (CORAL) for evaluating LLMs in oncological information extraction. The corpus contained 40 progress notes annotated hierarchically by oncology specialists, capturing entities, attributes, and relationships. Three models: GPT-4, GPT-3.5-turbo, and FLAN-UL2, were tested in a zero-shot setting on extracting clinical narrative elements such as symptoms, test results, and treatment plans. GPT-4 achieved the best overall performance, demonstrating proficiency in extracting tumour grade and inferring treatment progression, with manual evaluation confirming 68% accuracy. Nevertheless, hallucinations and inconsistent causal reasoning persisted, indicating that even state-of-the-art models are not yet suitable for direct clinical application. Moreover, the study did not explore the potential benefits of prompt engineering, leaving room for future enhancement.

Summary

Across these studies, LLMs consistently demonstrate strong capabilities in extracting clinical information from unstructured data, frequently outperforming traditional NLP and rule-based systems. Techniques such as PEFT, RAG, and carefully engineered prompts emerge as key factors for improving accuracy and reliability. However, limitations persist in terms of hallucination control, contextual reasoning, and semantic nuance understanding. The need for domain-specific fine-tuning, evaluation frameworks, and interpretability remains central to ensuring trustworthy clinical deployment.

Matching Patients for Clinical Trials with LLMs

The automation of patient–trial matching represents a major frontier in clinical informatics, aiming to accelerate recruitment, reduce operational costs, and ensure equitable access to medical research. Traditional matching pipelines are labor-intensive and constrained by the unstructured nature of electronic health records (EHRs). Recent studies have demonstrated that large language models (LLMs) can address these challenges by interpreting clinical narratives and eligibility criteria, offering explainability and scalability through zero-shot and few-shot learning. This section presents a synthesis of recent works exploring LLM-based frameworks for clinical trial matching.

Zero-Shot Matching and the TrialGPT Framework

The paper “Matching patients to clinical trials with large language models” introduces **TrialGPT**, a modular framework employing LLMs for zero-shot patient–trial matching. It consists of three interconnected modules:

1. **TrialGPT-Retrieval**, which filters a large pool of trials by extracting keywords from EHRs;
2. **TrialGPT-Matching**, which evaluates patient eligibility criterion-by-criterion, generating natural language explanations, localizing relevant EHR phrases, and producing an eligibility classification; and
3. **TrialGPT-Ranking**, which aggregates eligibility predictions at the trial level to rank potential matches.

Two ranking strategies were proposed: Linear Aggregation, based on direct percentage computations for inclusion and exclusion criteria, and LLM Aggregation, which leverages LLMs to infer global relevance and eligibility. TrialGPT-Retrieval recalled 90% of relevant trials using only 6% of the total dataset, with the generated keywords surpassing those manually produced by experts. TrialGPT-Matching achieved an 87.3% accuracy, nearly matching human specialists (0.887–0.900), and demonstrated an F1 score of 88.6% in identifying relevant phrases. The TrialGPT-Ranking module produced results highly correlated with expert judgments, outperforming competing systems by 43.8% in trial inclusion and exclusion classification. The study also estimated a 42.6% reduction in screening time, establishing TrialGPT as an efficient foundation for clinical trial automation.

Retrieval Pipelines and LLM-Based Re-Ranking

In “Learning to match patients to clinical trials using large language models,” a retrieval-based pipeline was developed to match patients with suitable clinical trials. The process began with query formulation, representing each patient’s profile as a query—using either a template-based or free-text representation generated by GPT-3.5-turbo. A BM25 model performed initial

retrieval of 1,000 candidate trials, indexing both individual and aggregated fields of trial descriptions. This was followed by a **re-ranking stage**, where several strategies were applied:

- **SCT-BERT**, a fine-tuned BERT variant specialized for clinical trial text.
- **TCRR**, inspired by curriculum learning, which first learns topical relevance and then fine-tunes on true eligibility.
- **LLM-based embeddings**, using semantic similarity between the patient and trial texts.
- **LLM-based relevance scoring**, where GPT-3.5-turbo directly produces numerical ranking scores.
- **LLM-based filtering**, applying binary yes/no filtering to the top candidates.

The results showed that fine-tuned LLM-based re-ranking achieved the highest retrieval precision, though at a substantial computational cost. GPT-4o demonstrated strong zero-shot performance comparable to fine-tuned GPT-3.5, albeit with higher latency. While Chain-of-Thought (CoT) prompting improved interpretability, it added computational overhead without clear accuracy gains. Pre-trained language model (PLM) re-rankers such as BERT offered a cost-efficient compromise, balancing performance and resource use. The study highlighted the persistent effectiveness–efficiency trade-off: while LLMs enhance matching quality, they remain computationally demanding, requiring significant inference time and large datasets for fine-tuning.

Ontology-Enhanced Prompt-Based Matching

The work “Towards efficient patient recruitment for clinical trials: application of a prompt-based learning model” proposed an approach combining LLMs with structured ontologies such as **SNOMED CT** to refine eligibility extraction. Using GPT-3.5-turbo in a deterministic configuration (temperature = 0), the system automatically annotated clinical records with MedCAT, linking text to SNOMED CT and UMLS terminologies. Phrases containing relevant concepts were extracted to reduce input length while preserving essential information. Each eligibility criterion was converted into a **question-based prompt** for binary classification (“yes” or “no”), forming a zero-shot evaluation pipeline.

The model achieved micro and macro F-measures of 0.9061 and 0.8060, respectively, placing it among the top-performing systems on the n2c2 dataset. Without summarization, the performance dropped considerably (micro: 0.7126; macro: 0.5277), underscoring the value of preprocessing and semantic abstraction. Limitations included high false-negative rates caused by non-standard abbreviations, ontology gaps (e.g., missing synonyms and street drug names), and class imbalance in certain criteria. Despite these challenges, the study demonstrated the viability of ontology-enhanced prompting for improving interpretability and precision in patient eligibility classification.

Distilling Open-Source Models for Patient–Trial Matching

The paper “Distilling Large Language Models for Matching Patients to Clinical Trials” presented the first systematic evaluation of **open-source versus proprietary LLMs** in this domain. The study compared GPT-3.5, GPT-4, and LLAMA-2 (7B, 13B, 70B) models, with a focus on refining open-source systems to rival proprietary performance. Each model performed multi-step reasoning: generating chain-of-thought explanations, identifying evidence phrases, and categorizing patients as “included,” “excluded,” or “no relevant information.” The trial ranking metric was computed as the difference between inclusion and exclusion fulfillment percentages.

To improve open-source adaptability, a synthetic dataset of 2,000 patient–trial pairs was generated by GPT-4, including explanations and evidence. LLAMA variants were fine-tuned using supervised loss on this dataset, resulting in the Trial-LLAMA 70B model. Evaluation metrics included NDCG@10, Precision@10, AUROC, and Criterion-Level Accuracy (CLA) for explicit and implicit reasoning, along with **Evidence Faithfulness** (precision, recall, and F1). GPT-4 remained the top-performing out-of-the-box model, while Trial-LLAMA 70B achieved performance parity with GPT-3.5 and even exceeded it in criterion-level accuracy after fine-tuning. The results confirmed that open-source models can achieve near-proprietary performance with task-aligned fine-tuning.

However, implicit reasoning remained challenging across all models, and the reliance on GPT-4-generated training data introduced potential bias. Furthermore, improvements in criterion-level accuracy did not always translate to higher trial-level ranking performance, revealing a non-linear relation between component accuracy and end-task success.

End-to-End Matching with Real-World Validation

The study “PRISM: Patient Records Interpretation for Semantic Clinical Trial Matching using Large Language Models” represents a significant advance, as it offers the **first empirical end-to-end validation** of a trial matching system using real-world unstructured EHRs. The PRISM framework formulates eligibility matching as a **question-answering problem**, where inclusion and exclusion criteria are decomposed into simplified questions with possible answers (“yes,” “no,” or “N/A”). Logical structures such as Disjunctive Normal Form (DNF) are employed to manage multi-question criteria.

PRISM introduces OncoLLM, a fine-tuned LLM based on Qwen-1.5 (14B parameters), specialized in oncological data and augmented with Chain-of-Thought reasoning. To handle long EHRs, the pipeline includes a **chunking and retrieval module** using *spaCy* for tokenization and chronological arrangement of patient notes. The final decision-making follows a **logical tree**, and probabilistic evaluation is used for missing data. Trials are ranked using a **Weighted Tier method** prioritizing clinically significant criteria.

OncoLLM achieved 63% criterion-level accuracy, approaching GPT-4’s 68%, while outperforming GPT-3.5-turbo and similarly sized models. It also excelled in both patient-centric and trial-centric rankings, with 95% of top-3 recommendations judged clinically appropriate and

a **35× cost reduction** compared to GPT-4. Despite its success, limitations included the exclusive use of unstructured data (omitting structured records such as lab results), dependence on imperfect retrievers, and annotation inconsistencies among human experts. These findings underline the need for hybrid systems combining structured and unstructured data with consistent expert validation.

Scalable Zero-Shot Matching Systems

Finally, “Zero-Shot Clinical Trial Patient Matching with LLMs” demonstrated the feasibility of scaling patient–trial matching through a **zero-shot architecture**. The system evaluated patient eligibility by comparing unstructured clinical histories to trial criteria using models such as GPT-4, GPT-3.5, Llama-2-70B-32k, and Mixtral-8x7B. Lightweight embedding models (BGE and MiniLM) were used for preliminary retrieval in a two-stage pipeline. Four prompting strategies were explored: All Criteria, All Notes (ACAN); All Criteria, Individual Notes (ACIN); Individual Criteria, All Notes (ICAN); and Individual Criteria, Individual Notes (ICIN); with **ACIN** identified as the most cost-efficient.

Using GPT-4 under the ACIN strategy, the system achieved state-of-the-art results on the n2c2 benchmark, with Micro-F1 = 0.93 and Macro-F1 = 0.81, surpassing prior rule-based systems and other LLMs. It also achieved dramatic cost reductions, screening 86 patients in two hours at only \$1.55 per patient. Clinician evaluation confirmed 97% coherence in model explanations. However, limitations included dataset simplification (English-only, limited criteria coverage), prompt dependence, and potential hallucinations. Privacy and compliance concerns also persist for proprietary models under healthcare regulations such as HIPAA.

Summary

Across all studies, LLMs demonstrate exceptional potential in automating patient-to-trial matching by leveraging zero-shot reasoning, retrieval-augmented architectures, and prompt-based interpretability. Fine-tuning and ontology integration notably improve performance, while hybrid retrieval pipelines balance speed and accuracy. Open-source models such as Trial-LLAMA and OncoLLM show that domain-specific fine-tuning can achieve or surpass proprietary benchmarks.

However, key challenges persist: high computational costs, sensitivity to prompt design, limited robustness in implicit reasoning, and difficulties in establishing reliable ground truths. Furthermore, real-world deployment must address privacy, interpretability, and integration with structured clinical data. Future research should focus on hybrid multimodal systems, explainability, and the harmonization of performance with efficiency to achieve clinically trustworthy deployment.